

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВГУ»)

Факультет компьютерных наук
Кафедра цифровых технологий

**Разработка интерактивной карты прогноза заболеваемости МКБ
(мочекаменной болезнью) в Воронежской области**

Отчет по учебной практике
(научно-исследовательская работа)

02.03.01 Математика и компьютерные науки

Профиль «Распределенные системы и искусственный интеллект»

Зав. кафедрой  Кургалин С. Д. д.ф.-м.н., профессор 12.06.2026

Обучающийся  Дорохова С. А.

Руководитель  Сирота Е. А. к.ф.-м.н., доцент

оценка: „отлично“

Воронеж 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Теоретические основы разработки интерактивных геоинформационных систем.....	6
1.1 Принципы работы с геопространственными данными в веб-приложениях.....	6
1.2 Обзор библиотек для визуализации карт в браузере (Leaflet, OpenLayers, Yandex Maps)	8
1.3 Формат GeoJSON и его роль в интеграции атрибутивных и геометрических данных	11
2 Методы и инструменты обработки данных для построения карты.....	14
2.1 Сбор и предобработка исходных медико-статистических данных с помощью Python (pandas)	14
2.2 Парсинг и верификация геопространственных границ районов Воронежской области.....	15
2.3 Методы прогнозирования показателей заболеваемости на основе временных рядов	16
3 Программная реализация интерактивного веб-модуля	18
3.1 Архитектура и структура программного модуля	18
3.2 Реализация клиентской части на JavaScript с использованием Leaflet.js	19
3.3 Генерация динамической HTML-страницы средствами Python	21
3.4 Интеграция всплывающих окон и фильтрации данных	22
4 Анализ результатов и оценка функциональности.....	25
4.1 Визуализация пространственного распределения прогнозных показателей	25
4.2 Проверка работоспособности и тестирование интерактивных элементов	26
4.3 Оценка масштабируемости и возможности адаптации для других регионов.....	27
ЗАКЛЮЧЕНИЕ	29
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	31
ПРИЛОЖЕНИЕ А	32

ВВЕДЕНИЕ

В настоящее время визуализация пространственно-распределённых данных является активно развивающейся областью веб-технологий. Существуют различные библиотеки и фреймворки для создания интерактивных карт: Leaflet.js, OpenLayers, Yandex Maps API, Google Maps API. Каждая из них имеет свои преимущества и недостатки. Однако большинство существующих решений ориентированы на ручное создание карт или требуют развёртывания серверной части. Вопросы автоматической генерации веб-карт на основе скриптовых языков (например, Python) с возможностью интеграции атрибутивных и геометрических данных остаются недостаточно проработанными. Кроме того, открытые геоданные для муниципальных образований зачастую отсутствуют в стандартизированном формате GeoJSON, что создаёт дополнительные трудности при разработке интерактивных карт для конкретных регионов.

Основанием для выполнения данной работы является необходимость создания программного инструмента, позволяющего автоматически генерировать интерактивные веб-карты на основе табличных статистических данных и геопространственных границ. Исходными данными для разработки послужили:

- статистические показатели заболеваемости мочекаменной болезнью по 32 муниципальным районам Воронежской области, а также по городским округам Воронеж и Нововоронеж (предоставлены в обезличенном виде);
- прогнозные значения заболеваемости на 2026 год, рассчитанные на основе временных рядов;
- геопространственные данные (границы районов), полученные в результате самостоятельного поиска из открытых картографических сервисов с последующей ручной верификацией и обработкой.

Актуальность работы обусловлена несколькими факторами. Во-первых, отсутствием готовых инструментов для автоматической генерации интерактивных карт на основе статистических данных, что особенно важно для региональных органов управления, не имеющих доступа к специализированным ГИС-решениям или не располагающих квалифицированным персоналом для их развёртывания. Во-вторых, сложностью получения и верификации геоданных в формате GeoJSON для конкретных муниципальных образований. В-третьих, необходимостью создания универсальных программных решений, которые могут быть адаптированы для различных предметных областей без изменения программного кода. Разработанный в работе веб-модуль позволяет решить эти проблемы, объединяя обработку данных на Python и интерактивную визуализацию на Leaflet.js.

Данная работа продолжает и развивает исследования в области применения веб-технологий для визуализации пространственно-распределённых данных. В частности, работа опирается на исследования Калинина П.В. [1], посвящённые пространственному анализу в геоинформатике, а также на работы Борисова В.В. и Ковалева В.А. [2] в области медицинской географии. Кроме того, использованы подходы и методы обработки геоданных и визуализации, изложенные в статье Дороховой С.А., Борисова А.И. и Сироты Е.А. [5], посвящённой разработке интерактивной карты прогноза заболеваемости в Воронежской области. Также в работе используются подходы и методы, описанные в официальной документации библиотеки Leaflet.js [3] и в руководствах по обработке формата GeoJSON с помощью Python. В отличие от указанных работ, в данном исследовании основное внимание уделяется автоматической генерации интерактивной веб-карты с использованием связки Python и JavaScript, а также интеграции атрибутивных и геометрических данных без необходимости установки дополнительного серверного программного обеспечения.

Целью работы является разработка интерактивного веб-модуля для визуализации пространственно-распределённых данных с возможностью фильтрации, детализации и анализа, апробированного на примере прогнозных показателей заболеваемости мочекаменной болезнью в Воронежской области.

Задачи исследования:

- провести анализ существующих инструментов и библиотек для создания интерактивных веб-карт (Leaflet.js, OpenLayers, Yandex Maps), обосновать выбор Leaflet.js для реализации;
- разработать процедуру сбора и предобработки исходных данных: атрибутивных (статистические показатели) и геометрических (границы районов в формате GeoJSON);
- реализовать программный модуль на Python, выполняющий обработку данных, расчёт прогнозных значений и генерацию HTML-страницы со встроенными данными;
- разработать клиентскую часть на JavaScript с использованием Leaflet.js для отображения карты, цветовой кодировки районов, всплывающих подсказок и информационных окон;
- провести тестирование разработанного модуля, оценить его функциональность и масштабируемость для других регионов.

Практическая значимость работы состоит в возможности использования разработанного модуля региональными министерствами здравоохранения для кластеризации зон риска, планирования выделения врачей-урологов и закупки оборудования.

1 Теоретические основы разработки интерактивных геоинформационных систем

1.1 Принципы работы с геопространственными данными в веб-приложениях

Геопространственные данные — это информация, имеющая явную или неявную привязку к определённому местоположению на поверхности Земли. Такие данные могут быть представлены в виде географических координат (широта, долгота), адресов, почтовых индексов или границ территорий. В веб-приложениях геопространственные данные используются для создания интерактивных карт, расчёта маршрутов, анализа территорий и визуализации пространственно-распределённых явлений. В контексте данной работы основным типом геопространственных данных являются **полигоны**, описывающие границы муниципальных районов Воронежской области.

Основными типами геопространственных данных являются векторные и растровые. **Векторные данные** представляют объекты в виде точек, линий и полигонов. Точка может обозначать населённый пункт, линия — дорогу или реку, полигон — границу района или области. Каждый векторный объект может содержать набор атрибутов (например, название, численность населения, уровень заболеваемости). **Растровые данные** представляют собой матрицы значений, где каждая ячейка (пиксель) содержит числовую информацию. Примером растровых данных являются спутниковые снимки, карты высот или температурные карты. В веб-приложениях наиболее часто используются векторные данные, так как они компактны, легко передаются по сети и позволяют гибко изменять стилизацию объектов.

Для передачи векторных геоданных в веб-среде был разработан формат GeoJSON, который является расширением стандарта JSON (JavaScript Object Notation) и предназначен для представления геометрических объектов вместе с их атрибутами. GeoJSON стал отраслевым стандартом и поддерживается большинством картографических библиотек, включая Leaflet.js, OpenLayers, Mapbox и Yandex Maps. В рамках данной работы

GeoJSON используется для хранения и передачи границ муниципальных районов Воронежской области.

Принцип работы веб-приложения с геопространственными данными обычно включает следующие этапы:

- 1) **сбор данных** — получение координат и атрибутов из источников (в данной работе исходные геоданные (границы районов) отсутствовали в открытом доступе в требуемом формате, что потребовало их самостоятельного сбора и верификации);
- 2) **обработка и очистка** — удаление дубликатов, исправление ошибок, приведение к единой системе координат;
- 3) **преобразование в формат GeoJSON** — структурирование данных в виде объектов, содержащих геометрию (координаты) и свойства (атрибуты);
- 4) **загрузка в веб-приложение** — передача GeoJSON-файла в браузер для последующей визуализации;
- 5) **интерактивная визуализация** — отображение геоданных на карте с использованием клиентских библиотек.

Важным аспектом работы с геопространственными данными является **система координат**. Для веб-приложений наиболее распространённой является система WGS84 (World Geodetic System 1984), в которой координаты задаются в градусах широты и долготы. Эта система используется в GPS и большинстве картографических сервисов, включая OpenStreetMap и Google Maps. Все геоданные, используемые в данной работе, приведены к системе WGS84.

Для эффективной работы с геопространственными данными в веб-приложениях необходимо учитывать следующие требования:

- 1) **производительность** — при работе с большими наборами данных необходимо оптимизировать их загрузку и рендеринг (например, использовать методы кластеризации или упрощения геометрии);

- 2) **точность** (координаты должны соответствовать реальным географическим объектам с допустимой погрешностью, ошибки в границах могут привести к некорректной визуализации);
- 3) **масштабируемость** (приложение должно корректно отображать данные на разных уровнях детализации (zoom));
- 4) **интерактивность** (пользователь должен иметь возможность взаимодействовать с объектами: кликать, наводить курсор, получать дополнительную информацию. В данной работе интерактивность реализована через всплывающие подсказки (tooltips) и информационные окна (pop-ups)).

Описанные принципы работы с геопространственными данными в веб-приложениях включают комплексные этапы от сбора и обработки данных до их интерактивной визуализации с использованием современных веб-технологий и стандартизированных форматов. В данной работе эти принципы реализованы на примере визуализации границ районов Воронежской области с последующей привязкой атрибутивных данных о заболеваемости.

1.2 Обзор библиотек для визуализации карт в браузере (Leaflet, OpenLayers, Yandex Maps)

Для создания интерактивных карт в веб-приложениях существует несколько популярных библиотек. Каждая из них имеет свои особенности, преимущества и ограничения. Выбор конкретной библиотеки зависит от требований к проекту: необходимости работы с большими объёмами данных, поддержки определённых форматов, производительности, наличия документации и лицензионных ограничений. В данном разделе рассматриваются три наиболее распространённые библиотеки: Leaflet.js, OpenLayers и Yandex Maps API.

Leaflet.js — это легковесная библиотека с открытым исходным кодом, предназначенная для создания интерактивных карт. Она была создана Владимиром Агафонкиным в 2010 году и с тех пор стала одним из самых

популярных инструментов для веб-картографии. Основные преимущества Leaflet.js:

- 1) небольшой размер (около 40 КБ в сжатом виде), что обеспечивает быструю загрузку страницы;
- 2) поддержка всех основных браузеров, включая мобильные;
- 3) расширяемость за счёт плагинов (тепловые карты, кластеризация, экспорт в изображения);
- 4) полная совместимость с форматом GeoJSON;
- 5) бесплатное использование без лицензионных ограничений (лицензия BSD).

OpenLayers — это мощная библиотека с открытым исходным кодом, ориентированная на решение сложных картографических задач. Она поддерживает работу с различными источниками данных (WMS, WMTS, KML, GeoJSON, OSM) и имеет встроенные инструменты для работы с проекциями и координатными системами. Однако OpenLayers имеет больший размер (около 200 КБ) и более сложный API по сравнению с Leaflet, что может замедлить разработку для небольших проектов.

Yandex Maps API — это коммерческая библиотека от компании Яндекс, предназначенная для создания карт на территории России и стран СНГ. Основные преимущества: высокое качество картографической подложки для российских регионов, наличие готовых решений для маршрутизации и геокодирования. Однако использование Yandex Maps API требует наличия ключа (API-ключа), а бесплатная версия имеет ограничения по количеству запросов и функциональности.

Для наглядного сравнения характеристик указанных библиотек приведена таблица 1.

Таблица 1 – Сравнение библиотек для создания интерактивных карт

Характеристика	Leaflet.js	OpenLayers	Yandex Maps API
----------------	------------	------------	-----------------

Окончание таблицы

Размер библиотеки	~40 КБ	~200 КБ	~100 КБ (с учётом API)
Лицензия	BSD (свободная)	BSD (свободная)	Коммерческая
Поддержка GeoJSON	Полная	Полная	Ограниченная
Производительность	Высокая	Средняя	Средняя
Поддержка мобильных устройств	Да	Да	Да
Необходимость API-ключа	Нет	Нет	Да
Сложность освоения	Низкая	Высокая	Средняя
Качество карт для России	Среднее	Среднее	Высокое

На рисунке 1 показан пример интерфейса карты, созданной с использованием Leaflet.js с подключённым GeoJSON-слоем. На карте отображены полигоны районов, окрашенные в зависимости от числового значения атрибута (реализация цветовой кодировки).

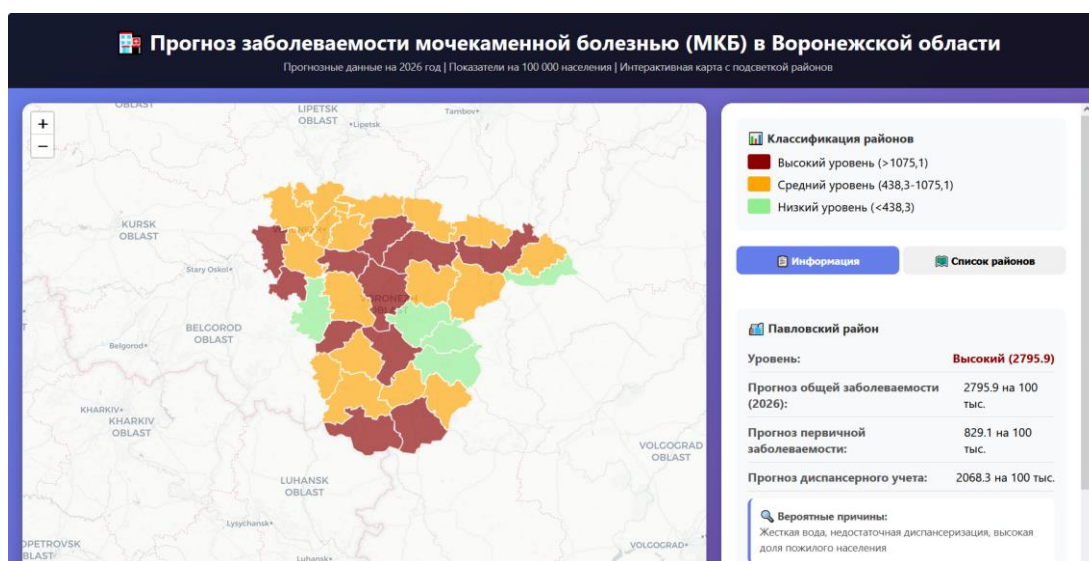


Рисунок 1 – Пример карты Leaflet.js с цветовой кодировкой полигонов

На основе проведенного сравнения для реализации программного модуля была выбрана библиотека Leaflet.js. Этот выбор обусловлен следующими причинами:

- 1) **работа с GeoJSON** (Leaflet.js обеспечивает полную поддержку формата GeoJSON, который используется в данной работе для хранения границ

районов. Это позволяет легко интегрировать геометрические и атрибутивные данные);

- 2) **автоматическая генерация HTML-страницы** (поскольку в работе используется Python для генерации HTML-кода, лёгкость встраивания Leaflet.js в статическую страницу является ключевым преимуществом);
- 3) **отсутствие API-ключа** (в отличие от Yandex Maps API, Leaflet.js не требует регистрации и получения лицензионных ключей, что упрощает развёртывание и распространение приложения);
- 4) **производительность** (небольшой размер библиотеки обеспечивает быструю загрузку страницы, что важно для веб-приложений, особенно при работе с большим количеством полигонов);
- 5) **простота освоения** (низкий порог входа позволяет быстро реализовать требуемый функционал (цветовая кодировка, всплывающие окна, фильтрация) без изучения сложного API);
- 6) **расширяемость** (при необходимости функционал карты может быть расширен за счёт плагинов (например, для экспорта в PDF или отображения тепловых карт)).

Таким образом, Leaflet.js является оптимальным выбором для разработки интерактивной карты в рамках данной работы, что подтверждается её широким использованием в научных и коммерческих проектах, а также наличием обширной документации и активного сообщества разработчиков.

1.3 Формат GeoJSON и его роль в интеграции атрибутивных и геометрических данных

Формат GeoJSON является открытым стандартом для представления пространственных данных, основанным на JSON (JavaScript Object Notation). Он был разработан для обмена географическими данными в веб-среде и поддерживается большинством картографических библиотек, включая Leaflet.js. Главное преимущество GeoJSON заключается в возможности одновременного хранения геометрической информации (координаты, границы

объектов) и атрибутивной информации (свойства объекта, например, название района, числовые показатели).

В контексте данной работы основным типом геометрических объектов являются полигоны (Polygon), описывающие границы муниципальных районов. Каждый полигон в формате GeoJSON представляется в виде массива координат, где каждая координата задаётся парой чисел [долгота, широта]. Для работы с данными, имеющими атрибуты, используется тип Feature, который объединяет геометрию и свойства. Структура такого Feature-объекта, взятого из используемого в данной работе GeoJSON-файла, приведена в **Листинге А.1** (см. Приложение А). Полный GeoJSON-файл представляет собой FeatureCollection — коллекцию таких Feature-объектов. На рисунке 2 показан пример структуры GeoJSON-файла, используемого в данной работе.

Полный GeoJSON-файл представляет собой **FeatureCollection** — коллекцию таких Feature-объектов. На рисунке 2 показан пример структуры GeoJSON-файла, используемого в данной работе.

```
1 {
2   "type": "FeatureCollection",
3   "features": [
4     { "type": "Feature", "properties": { "district": "Аннинский район", "geometry": { "type": "Polygon", "coordinates": [ [ [ 40.0572322, 51.325726 ], [ 40.0584986, 51.3254
5     { "type": "Feature", "properties": { "district": "Бобровский район", "geometry": { "type": "Polygon", "coordinates": [ [ [ 39.6871295, 51.1831117 ], [ 39.7542491, 51.1597559
6     { "type": "Feature", "properties": { "district": "Богучарский район", "geometry": { "type": "Polygon", "coordinates": [ [ [ 40.1713309, 49.9667572 ], [ 40.1791279, 49.9558504
7     { "type": "Feature", "properties": { "district": "Борисоглебский район", "geometry": { "type": "Polygon", "coordinates": [ [ [ 41.877752, 51.3055489 ], [ 41.9090636, 51.28994
8     { "type": "Feature", "properties": { "district": "Бутурлиновский район", "geometry": { "type": "Polygon", "coordinates": [ [ [ 40.0947974, 50.8837276 ], [ 40.0990845, 5
9     { "type": "Feature", "properties": { "district": "Верхнемамонский район", "geometry": { "type": "Polygon", "coordinates": [ [ [ 39.9987875, 50.1107531 ], [ 39.9992648,
10    { "type": "Feature", "properties": { "district": "Верхнехавский район", "geometry": { "type": "Polygon", "coordinates": [ [ [ 39.3525805, 51.9076016 ], [ 39.3537484, 51
11    { "type": "Feature", "properties": { "district": "Воробьевский район", "geometry": { "type": "Polygon", "coordinates": [ [ [ 40.6062699, 50.5870707 ], [ 40.6123515, 50
12    { "type": "Feature", "properties": { "district": "Грибановский район", "geometry": { "type": "Polygon", "coordinates": [ [ [ 41.1230516, 51.5248501 ], [ 41.1372181, 51
13    { "type": "Feature", "properties": { "district": "Калачеевский район", "geometry": { "type": "Polygon", "coordinates": [ [ [ 40.4974446, 50.4497606 ], [ 40.497806, 50
14    { "type": "Feature", "properties": { "district": "Каменинский район", "geometry": { "type": "Polygon", "coordinates": [ [ [ 39.0457159, 50.6186486 ], [ 39.062554, 50.613
15    { "type": "Feature", "properties": { "district": "Кантемировский район", "geometry": { "type": "Polygon", "coordinates": [ [ [ 39.195453, 49.873309 ], [ 39.20214, 49.87
16    { "type": "Feature", "properties": { "district": "Каширский район", "geometry": { "type": "Polygon", "coordinates": [ [ [ 39.0866227, 51.367242 ], [ 39.0866778, 51.3649
17    { "type": "Feature", "properties": { "district": "Лискинский район", "geometry": { "type": "Polygon", "coordinates": [ [ [ 39.1360485, 51.0518777 ], [ 39.1363421, 51.05
18    { "type": "Feature", "properties": { "district": "Нижнедевичий район", "geometry": { "type": "Polygon", "coordinates": [ [ [ 38.1377516, 51.632316 ], [ 38.1388378, 51
19    { "type": "Feature", "properties": { "district": "Новоусманский район", "geometry": { "type": "Polygon", "coordinates": [ [ [ 39.2376637, 51.4917673 ], [ 39.2627459, 51
20    { "type": "Feature", "properties": { "district": "Новооперский район", "geometry": { "type": "Polygon", "coordinates": [ [ [ 41.0065863, 50.9844206 ], [ 41.0096566, 50
21    { "type": "Feature", "properties": { "district": "Ольховатский район", "geometry": { "type": "Polygon", "coordinates": [ [ [ 38.9814478, 50.4386177 ], [ 38.9951747, 50
22    { "type": "Feature", "properties": { "district": "Острогожский район", "geometry": { "type": "Polygon", "coordinates": [ [ [ 38.706536, 50.8002035 ], [ 38.7078105, 50
23    { "type": "Feature", "properties": { "district": "Павловский район", "geometry": { "type": "Polygon", "coordinates": [ [ [ 39.7596637, 50.7453182 ], [ 39.7601945, 50
24    { "type": "Feature", "properties": { "district": "Панинский район", "geometry": { "type": "Polygon", "coordinates": [ [ [ 39.6707798, 51.4700725 ], [ 39.7075595, 51
25    { "type": "Feature", "properties": { "district": "Петропавловский район", "geometry": { "type": "Polygon", "coordinates": [ [ [ 40.6332351, 50.1215486 ], [ 40.6360811,
26    { "type": "Feature", "properties": { "district": "Поворинский район", "geometry": { "type": "Polygon", "coordinates": [ [ [ 41.8024851, 51.2571668 ], [ 41.8473974, 51
27    { "type": "Feature", "properties": { "district": "Подгоренский район", "geometry": { "type": "Polygon", "coordinates": [ [ [ 39.2714237, 50.5207209 ], [ 39.2767532, 50
28    { "type": "Feature", "properties": { "district": "Рамонский район", "geometry": { "type": "Polygon", "coordinates": [ [ [ 38.7332894, 51.9951354 ], [ 38.7668719, 51
29    { "type": "Feature", "properties": { "district": "Рельевский район", "geometry": { "type": "Polygon", "coordinates": [ [ [ 38.3803527, 51.1502075 ], [ 38.3804575, 51
30    { "type": "Feature", "properties": { "district": "Россошанский район", "geometry": { "type": "Polygon", "coordinates": [ [ [ 39.1774939, 49.8978643 ], [ 39.1790739, 49
```

Рисунок 2 – Пример структуры GeoJSON-файла с данными о районах Воронежской области

В данной работе GeoJSON используется для интеграции двух типов данных: геометрических (границы районов) и атрибутивных (прогнозные показатели заболеваемости). Исходный GeoJSON-файл загружается в Python, затем к каждому Feature-объекту добавляются атрибутивные данные из

словаря `districts_info`. После этого обновлённый GeoJSON используется для генерации HTML-страницы с интерактивной картой.

Одной из проблем при интеграции геоданных является несоответствие названий районов в разных источниках, для решения которой применяются алгоритмы нормализации и частичного сопоставления строк (подробнее см. раздел 2.2). Использование GeoJSON в данной работе обеспечивает следующие преимущества: совместимость с Leaflet.js, простоту обработки с помощью Python, возможность расширения атрибутов и компактность данных.

Таким образом, GeoJSON является оптимальным форматом для интеграции геометрических и атрибутивных данных в веб-приложениях и используется в данной работе как основной формат для хранения границ районов Воронежской области.

2 Методы и инструменты обработки данных для построения карты

2.1 Сбор и предобработка исходных медико-статистических данных с помощью Python (pandas)

Для разработки интерактивной карты необходимы два типа данных: атрибутивные (числовые показатели) и геометрические (границы районов). В данном разделе описывается процесс сбора и предобработки атрибутивных данных с использованием языка Python и библиотеки pandas.

Исходными данными послужили анонимизированные статистические показатели заболеваемости мочекаменной болезнью по 32 муниципальным районам Воронежской области, а также по городским округам Воронеж и Нововоронеж. Данные включали общую заболеваемость, первичную заболеваемость и показатели диспансерного учёта (все — на 100 тыс. населения). Особенностью исходных данных являлось наличие пропусков и аномальных значений (например, отрицательные значения первичной заболеваемости для некоторых районов), что потребовало предобработки.

Для обработки данных использовался язык Python с библиотеками pandas (загрузка, очистка, преобразование), numpy (математические операции) и json (сохранение словарей). Процесс предобработки включал загрузку данных, удаление дубликатов, обработку пропусков, коррекцию аномальных значений (отрицательные значения приводились к нулю), формирование словарей данных и их сохранение в формате JSON.

В результате были получены три словаря:

- `mkb_forecast_2026` — прогноз общей заболеваемости на 2026 год;
- `primary_forecast_2026` — прогноз первичной заболеваемости на 2026 год;
- `dispensary_forecast_2026` — прогноз диспансерного учёта на 2026 год.

Каждый словарь содержит 34 записи (32 района + 2 города). Данные проверены на корректность: аномалии отсутствуют, суммарные показатели соответствуют ожидаемым значениям. Полный код, содержащий подготовку

этих словарей и настройку пороговых значений уровней заболеваемости, приведён в **Листинге А.2** Приложения А.

2.2 Парсинг и верификация геопространственных границ районов Воронежской области

Для создания интерактивной карты необходимы не только атрибутивные данные (числовые показатели), но и геометрические — границы муниципальных районов в формате GeoJSON.

Геопространственные данные (границы районов Воронежской области) в формате GeoJSON отсутствовали в открытом доступе в стандартизированном виде. Готовые файлы из официальных источников либо не содержали нужной информации, либо были представлены в форматах, несовместимых с Leaflet.js. Таким образом, возникла необходимость самостоятельного поиска и последующей обработки геоданных.

Для получения границ районов использовались открытые картографические сервисы и веб-инструменты для экспорта границ (например, GeoJSON.io), а также специализированные репозитории с геоданными. Поиск был затруднён тем, что большинство доступных файлов содержали границы либо всей Воронежской области, либо её крупных частей, но не отдельные районы. В результате был найден файл, содержащий границы районов, однако он требовал дополнительной обработки и верификации.

После получения файла с границами районов потребовалось проверить его точность и соответствие реальным административным границам. Верификация включала визуальную проверку (наложение полигонов на карту и сравнение с официальными источниками), сравнение с эталонными данными и коррекцию ошибок (например, пересечение границ, отсутствие некоторых районов). Ошибки в геоданных могут привести к некорректной визуализации на карте, поэтому верификация была выполнена с высокой степенью тщательности.

После верификации границ необходимо было сопоставить каждый полигон (район) с атрибутивными данными из словарей, полученных на этапе

2.1. Для этого использовалась библиотека `geopandas`. Основная сложность на этом этапе — несоответствие названий районов в GeoJSON-файле и в словарях данных (например, «Богучарский район» и «Богучарский»). Для решения этой проблемы была написана функция `matchDistrictName`, которая выполняет нормализацию названий и находит соответствие по частичному совпадению. Реализация функции `matchDistrictName`, а также код загрузки и сопоставления GeoJSON-данных представлены в **Листинге А.3** (см. Приложение А).

В результате поиска, обработки и верификации был получен корректный GeoJSON-файл, содержащий границы всех 32 районов Воронежской области и городов Воронеж и Нововоронеж. Каждый объект в файле содержит как геометрию (полигон), так и атрибутивные данные (название, прогнозные показатели заболеваемости, цветовую кодировку). Файл был сохранён и использован для генерации интерактивной карты.

2.3 Методы прогнозирования показателей заболеваемости на основе временных рядов

Для создания интерактивной карты недостаточно отобразить текущие статистические данные — необходимо также предоставить пользователю информацию о прогнозных значениях на будущий период. В данном разделе описывается метод прогнозирования, применённый для расчёта показателей заболеваемости на 2026 год.

Прогнозирование выполнялось на основе **временных рядов** — последовательности значений показателей заболеваемости, упорядоченных по годам. Для каждого района Воронежской области был построен временной ряд по данным за предшествующие годы. Затем на основе этих рядов рассчитывались прогнозные значения на 2026 год.

В качестве метода прогнозирования была выбрана **линейная экстраполяция трендов**. Данный метод предполагает, что изменение показателя во времени подчиняется линейному закону (1):

$$y = a \times t + b, \quad (1)$$

где y — прогнозируемый показатель, t — временной период (год), a и b — коэффициенты линейной регрессии, определяемые методом наименьших квадратов. Для каждого района коэффициенты a и b рассчитывались на основе ретроспективных данных, после чего вычислялось значение y для $t = 2026$.

Реализация прогнозирования выполнена на языке Python с использованием библиотек `numpy` (для вычисления коэффициентов регрессии) и `pandas` (для работы с временными рядами). Для каждого района была написана функция, которая принимает на вход массив значений за предыдущие годы и возвращает прогнозное значение.

После расчёта прогнозных значений для всех районов была выполнена **классификация уровней заболеваемости** на основе статистического анализа распределения полученных значений. Пороговые значения определены следующим образом:

- **высокий уровень** — значения выше 1075,1 на 100 тыс. населения (превышают среднее по совокупности районов на одно стандартное отклонение);
- **средний уровень** — значения в диапазоне от 438,3 до 1075,1;
- **низкий уровень** — значения ниже 438,3.

Такая классификация позволяет визуально выделить на карте зоны с различной эпидемиологической ситуацией и облегчает интерпретацию данных для конечного пользователя.

В результате применения описанного метода были получены прогнозные значения для всех 32 районов Воронежской области и городских округов Воронеж и Нововоронеж. Эти значения были сохранены в словарях `mkb_forecast_2026`, `primary_forecast_2026` и `dispensary_forecast_2026`, которые впоследствии использовались при генерации интерактивной карты (раздел 3).

3 Программная реализация интерактивного веб-модуля

3.1 Архитектура и структура программного модуля

Разработанный программный модуль представляет собой совокупность взаимосвязанных компонентов, обеспечивающих полный цикл обработки данных и генерации интерактивной веб-карты. Архитектура построена по принципу разделения ответственности: сбор и обработка данных выполняются на стороне сервера (Python), а визуализация и взаимодействие с пользователем — на стороне клиента (JavaScript). Такой подход позволяет добиться автономности готового решения: для работы с картой не требуется установка дополнительного серверного ПО или развёртывание базы данных.

Основными компонентами модуля являются: модуль обработки данных на Python, клиентский модуль на JavaScript с библиотекой Leaflet.js, а также связующий компонент — сгенерированная HTML-страница. Модуль обработки данных выполняет загрузку атрибутивных (статистические показатели) и геометрических (границы районов в GeoJSON) данных, их сопоставление, расчёт прогнозных значений и генерацию HTML-страницы со встроенным JSON-кодом. Клиентский модуль обеспечивает отображение карты, цветовую кодировку районов, всплывающие подсказки, информационные окна и боковую панель со списком районов.

Взаимодействие компонентов происходит в четыре этапа. На первом этапе Python-скрипт загружает исходные данные и с помощью функции `matchDistrictName` сопоставляет названия районов. На втором этапе выполняется расчёт прогнозных значений и классификация уровней заболеваемости. На третьем этапе на основе обработанных данных генерируется HTML-страница (см. рисунок 3), содержащая встроенный JavaScript-код, JSON-данные и элементы интерфейса. На четвёртом этапе страница открывается в браузере, где Leaflet.js отображает интерактивную карту.

```
✓ Найден файл: Voronezh_obl.geojson.txt

=====
🗺 КЛАССИФИКАЦИЯ РАЙОНОВ ПО ПРОГНОЗУ НА 2026 ГОД
=====
• 🔴 Высокий уровень (>1075.1): 11 районов
• 🟡 Средний уровень (438.3-1075.1): 18 районов
• 🟢 Низкий уровень (<438.3): 5 районов
• 🏠 Воронеж: Средний уровень (950.0)
• 🏠 Нововоронеж: Средний уровень (850.0)
=====

✓ Карта сохранена: D:\VSU\KURSOVAYA_3KURS\KODE\voronezh_mkb_forecast_2026.html

📌 Добавленные функции:
• 🟡 Подсветка районов при наведении (жёлтая обводка)
• 🗺 Отображение границ районов с цветовой заливкой
• 💬 Всплывающие подсказки при наведении
• 📍 Клик по району показывает подробную информацию
• 🏠 Добавлены города Воронеж и Нововоронеж (средний уровень)

⚠ ВНИМАНИЕ: Файл Воронежская область_Voronezh.txt не найден!
Карта не сможет отобразить границы районов.
Убедитесь, что файл находится в той же папке.

🌐 Файл автоматически открыт в браузере!

Process finished with exit code 0
```

Рисунок 3 – Генерация HTML-страницы в консоли Python

Предложенная архитектура обеспечивает автономность, переносимость и масштабируемость: для работы достаточно открыть HTML-файл в браузере, все данные упакованы в один файл, а замена исходных данных позволяет адаптировать модуль для других регионов без изменения кода. Эти преимущества делают разработанный модуль готовым к практическому применению.

3.2 Реализация клиентской части на JavaScript с использованием Leaflet.js

Клиентская часть разработанного программного модуля реализована на языке JavaScript с использованием библиотеки Leaflet.js. Этот компонент отвечает за отображение интерактивной карты, визуализацию геоданных и обеспечение взаимодействия с пользователем. Все функции клиентской части встроены непосредственно в сгенерированную HTML-страницу.

После загрузки страницы в браузере инициализируется экземпляр карты с помощью функции `L.map()`. Начальный центр карты устанавливается в

координаты Воронежской области, а масштаб подбирается таким образом, чтобы были видны все районы. Затем с помощью метода `L.tileLayer()` добавляется базовый картографический слой, предоставляемый сервисом `OpenStreetMap`.

Основным элементом клиентской части является слой `GeoJSON`, содержащий границы районов. Для его добавления используется метод `L.geoJson()`, в который передаётся объект с геоданными, а также параметры стилизации и обработки событий. Стилизация выполняется с помощью функции `styleFeature`, которая определяет цвет заливки полигона в зависимости от уровня заболеваемости района: красный для высокого уровня, оранжевый для среднего, зелёный для низкого. Цвета задаются в формате HEX и хранятся в словаре `districtsData`.

Для обеспечения интерактивности реализованы следующие функции. Функция `highlightFeature` при наведении курсора на район утолщает его границы и окрашивает их в жёлтый цвет, что помогает пользователю визуально выделить интересующий объект. Функция `resetHighlight` возвращает стиль района к исходному состоянию при уходе курсора. Функция `onFeatureClick` при клике на район получает его название, находит соответствующие данные в словаре `districtsData` и вызывает функцию `showDistrictInfo`, которая заполняет информационную панель подробными сведениями: уровень заболеваемости, прогноз общей и первичной заболеваемости, данные диспансерного учёта и предположительные причины. Эти функции связываются с каждым полигоном через параметр `onEachFeature` метода `L.geoJson()`.

Кроме того, на карте реализованы всплывающие подсказки (`tooltips`), которые появляются при наведении на район и отображают его название и уровень заболеваемости. Это позволяет пользователю быстро оценить ситуацию, не совершая лишних кликов. Всплывающие подсказки создаются с помощью метода `layer.bindTooltip()` и настраиваются на отображение в верхней части полигона.

Для удобства навигации и анализа данных в клиентской части также реализована боковая панель. Она содержит список всех районов, сгруппированных по уровням заболеваемости (высокий, средний, низкий). При клике на название района в списке карта автоматически центрируется на нём, а в информационной панели отображаются соответствующие данные. Это достигается путём вызова функции `showDistrictInfo` и последующего использования методов `map.flyTo()` и `layer.openTooltip()`.

Клиентская часть, реализованная на JavaScript с библиотекой Leaflet.js (см. **Листинг А.4** Приложения А), обеспечивает полную интерактивность модуля: визуализацию данных, подсветку при наведении, информационные окна при клике, всплывающие подсказки и фильтрацию через боковую панель. Это позволяет пользователю эффективно взаимодействовать с картой и получать всю необходимую информацию.

3.3 Генерация динамической HTML-страницы средствами Python

Генерация HTML-страницы является связующим звеном между обработкой данных на Python и визуализацией на JavaScript. Этот этап выполняется с помощью Python-скрипта (см. **Листинг А.5** Приложения А), который объединяет все подготовленные данные и формирует готовую к открытию в браузере HTML-страницу. Такой подход позволяет избежать необходимости развёртывания серверного ПО или использования баз данных.

Процесс генерации начинается после того, как завершены все этапы обработки данных: загружены и очищены атрибутивные показатели, получены и верифицированы геометрические границы районов, рассчитаны прогнозные значения и выполнена классификация уровней заболеваемости. На этом этапе все подготовленные данные уже хранятся в виде словарей Python (`mkb_forecast_2026`, `primary_forecast_2026`, `dispensary_forecast_2026`) и GeoJSON-объекта.

Формирование HTML-страницы выполняется с помощью f-строк Python. Создаётся многострочная строка, в которую вставляются все

необходимые компоненты. В первую очередь в шаблон включаются ссылки на внешние библиотеки: CSS-стили Leaflet.js и сам JavaScript-файл Leaflet.js, загружаемые из CDN. Затем в шаблоне определяются стили CSS для оформления карты, боковой панели, легенды и элементов интерфейса.

Далее в HTML-шаблон встраиваются данные в формате JSON. Для этого словари данных преобразуются в JSON-строки с помощью функции `json.dumps()` и сохраняются в JavaScript-переменные. Например, переменная `districtsData` содержит все атрибутивные данные о районах, а переменная `geojsonData` — геометрические границы. После этого в шаблоне размещается JavaScript-код, который при загрузке страницы создаёт карту Leaflet.js, загружает GeoJSON-слой, настраивает стилизацию и обработку событий.

После формирования полной HTML-строки она записывается в файл с расширением `.html` с помощью метода `write_text()`. В результате создаётся автономный HTML-файл, который можно открыть в любом современном браузере без дополнительных требований.

Преимуществами описанного подхода являются: автономность (не требуется сервер), переносимость (один файл), простота распространения (можно передать по электронной почте) и возможность быстрой адаптации под другие регионы путём замены исходных данных без изменения программного кода.

Генерация HTML-страницы средствами Python является ключевым этапом, объединяющим все компоненты разработанного модуля в единый готовый продукт.

3.4 Интеграция всплывающих окон и фильтрации данных

Интеграция всплывающих окон (pop-ups) и фильтрации данных является ключевым элементом, обеспечивающим удобство работы с картой для конечного пользователя. В разработанном модуле эти функции реализованы на клиентской стороне с использованием JavaScript и библиотеки Leaflet.js.

Всплывающие окна (pop-ups) реализованы следующим образом. При клике на полигон района вызывается функция `onFeatureClick`, которая определяет название района по его свойствам, находит соответствующий объект в словаре `districtsData` и заполняет информационную панель следующими данными: название района, уровень заболеваемости, прогноз общей заболеваемости, прогноз первичной заболеваемости, прогноз диспансерного учёта, предположительные причины заболеваемости. Информационная панель (см. рисунок 4) расположена справа от карты и автоматически обновляется при клике на любой район.

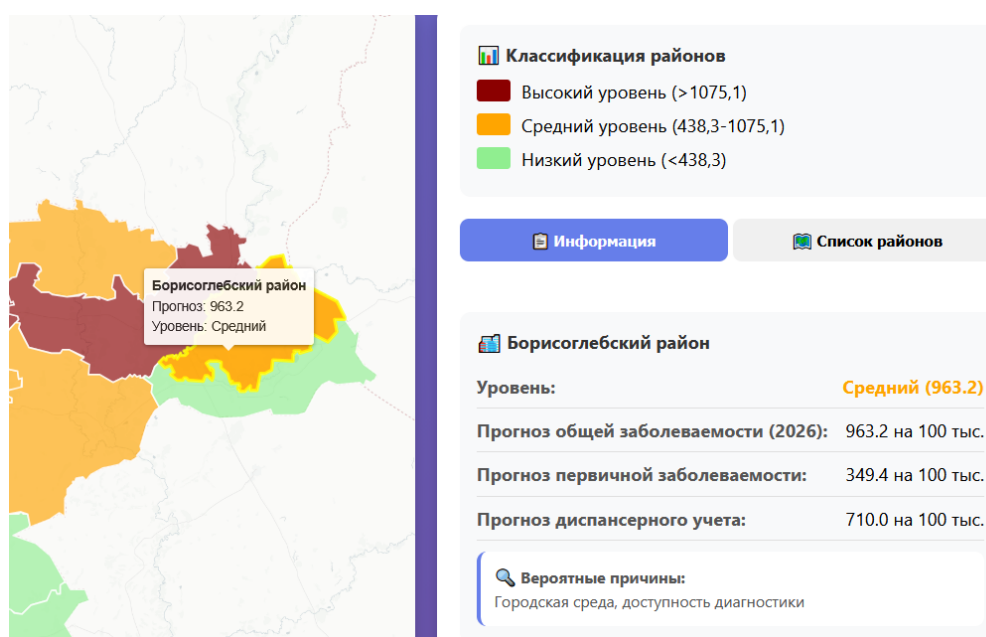


Рисунок 4 – Пример всплывающего окна с информацией о районе

Для реализации фильтрации данных в боковой панели создаётся три группы списков: «Высокий уровень», «Средний уровень» и «Низкий уровень». В каждой группе районы отсортированы по убыванию заболеваемости, что позволяет быстро выявить наиболее проблемные территории. При клике на название района из списка карта автоматически центрируется на нём с помощью метода `map.flyTo()`, а в информационной панели отображаются соответствующие данные. Это реализовано через функцию `showDistrictInfo`, которая вызывается при клике на элемент списка.

Дополнительно реализована возможность просмотра списка районов с группировкой по уровню риска. Пользователь может переключаться между вкладками «Информация» и «Список районов» в боковой панели. Вкладка «Список районов» (см. рисунок 5) отображает все районы, сгруппированные по уровням, с указанием прогнозного значения заболеваемости. При наведении на элемент списка он подсвечивается, а при клике — происходит переход к соответствующему району на карте.

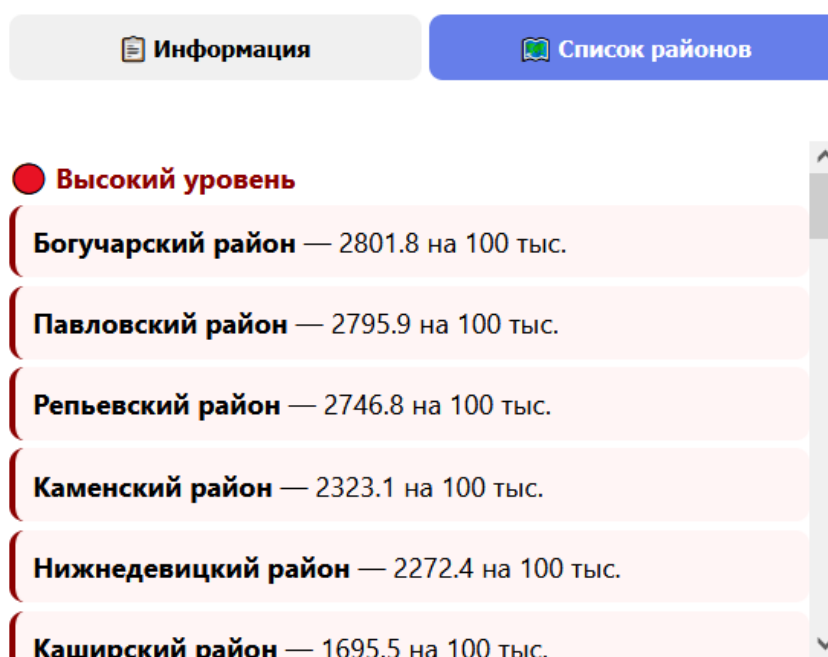


Рисунок 5 – Боковая панель со списком районов, сгруппированных по уровню риска

Интеграция всплывающих окон и фильтрации данных превращает разработанный модуль из простого визуализатора в полноценный инструмент анализа. Это делает его пригодным для практического использования в региональных системах здравоохранения, где требуется оперативная оценка эпидемиологической ситуации.

4 Анализ результатов и оценка функциональности

4.1 Визуализация пространственного распределения прогнозных показателей

В результате выполнения программного модуля была сгенерирована интерактивная карта, отражающая пространственное распределение прогнозных показателей заболеваемости мочекаменной болезнью на 2026 год. Карта содержит границы всех 32 муниципальных районов Воронежской области, а также городских округов Воронеж и Нововоронеж. Каждый район окрашен в соответствии с уровнем прогнозной заболеваемости: красным цветом отмечены районы с высоким уровнем (значения выше 1075,1 на 100 тыс. населения), оранжевым — со средним уровнем (от 438,3 до 1075,1), зелёным — с низким уровнем (менее 438,3). Такая цветовая кодировка позволяет быстро оценить эпидемиологическую ситуацию и выделить зоны риска.

Анализ полученной карты позволяет выделить следующие закономерности. Высокий уровень заболеваемости зафиксирован в 11 районах, преимущественно расположенных в южной и юго-западной частях области. К ним относятся Богучарский (2801,8), Павловский (2795,9), Репьевский (2746,8), Каменский (2323,1), Нижнедевицкий (2272,4), Каширский (1695,5), Аннинский (1234,9), Кантемировский (1219,0), Панинский (1201,7), Грибановский (1131,6) и Бобровский (1102,2). Средний уровень наблюдается в 18 районах, включая Воронеж (950,0) и Нововоронеж (850,0). Низкий уровень характерен для 5 районов: Бутурлиновский (159,0), Воробьевский (165,3), Калачеевский (301,8), Поворинский (299,8) и Острогожский (334,2).

Карта наглядно демонстрирует неравномерность распределения заболеваемости на территории Воронежской области. Наибольшие значения прогнозных показателей сконцентрированы в южных и юго-западных районах, что подтверждает актуальность использования пространственного анализа для планирования медицинских ресурсов и проведения профилактических мероприятий. Пользователь карты может в любой момент получить детальную

информацию о каждом районе, кликнув на него, а также воспользоваться фильтрацией через боковую панель для быстрого поиска интересных территорий.

4.2 Проверка работоспособности и тестирование интерактивных элементов

Тестирование разработанного программного модуля проводилось на локальной рабочей станции путём открытия сгенерированного HTML-файла в браузерах Google Chrome и Яндекс.Браузер. Целью тестирования являлась проверка корректности отображения всех элементов карты и работоспособности интерактивных функций, реализованных в клиентской части.

В ходе тестирования были проверены следующие функции:

- **отображение всех районов на карте** — проверка отсутствия пропущенных полигонов, а также корректность их геометрических границ;
- **цветовая кодировка** — соответствие цвета заливки каждого района его уровню заболеваемости (красный — высокий, оранжевый — средний, зелёный — низкий);
- **всплывающие подсказки (tooltips)** — при наведении курсора на район отображается его название и прогнозное значение заболеваемости;
- **информационные окна (pop-ups)** — при клике на район открывается окно с подробными данными: уровень заболеваемости, прогноз общей и первичной заболеваемости, диспансерный учёт, а также предположительные причины;
- **фильтрация и навигация через боковую панель** — при клике на название района в списке карта автоматически центрируется на нём и отображается соответствующая информация;
- **адаптивность интерфейса** — корректное отображение карты и элементов управления на экранах различного разрешения.

Все перечисленные функции были проверены и показали корректную работу. Ошибок в работе модуля не выявлено. Переходы между районами при клике на список происходят плавно, информационные окна содержат все запланированные данные, цветовая кодировка соответствует классификации. Карта успешно открывается в различных браузерах без необходимости установки дополнительных плагинов или расширений.

Разработанный модуль является полностью работоспособным и готовым к использованию.

4.3 Оценка масштабируемости и возможности адаптации для других регионов

Одним из ключевых преимуществ разработанного программного модуля является его масштабируемость, то есть возможность адаптации для других регионов Российской Федерации без изменения программного кода. Архитектура модуля построена таким образом, что все исходные данные (геометрические границы и атрибутивные показатели) отделены от логики обработки и визуализации. Это позволяет переносить инструмент на любую другую территорию путём замены входных файлов.

Для адаптации модуля к новому региону необходимо выполнить три простых шага:

- 1) **замена геопространственных данных** (вместо GeoJSON-файла с границами районов Воронежской области подставляется аналогичный файл, содержащий границы муниципальных образований целевого региона. Формат файла должен оставаться неизменным — GeoJSON с полигонами);
- 2) **обновление атрибутивных данных** (исходные словари с прогнозными значениями заболеваемости заменяются на актуальные статистические данные для нового региона. Структура словарей (название района — числовое значение) сохраняется);
- 3) **настройка пороговых значений** (при необходимости уровни классификации (высокий, средний, низкий) могут быть переопределены

в соответствии с региональными нормативными показателями. Это выполняется путём изменения значений констант LEVEL_HIGH_MIN, LEVEL_MEDIUM_MIN и LEVEL_MEDIUM_MAX в Python-скрипте).

Важно отметить, что процесс адаптации не требует изменения логики обработки данных или клиентской части на JavaScript. Все функции интерактивной карты (цветовая кодировка, всплывающие подсказки, информационные окна, фильтрация) будут работать автоматически для любого региона, для которого подготовлены корректные входные данные.

Таким образом, разработанный инструмент не является узкоспециализированным решением только для Воронежской области, а представляет собой универсальную архитектуру для визуализации медико-статистических данных, которая может быть внедрена в региональных системах здравоохранения по всей стране.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсовой работы был разработан программный модуль, интегрирующий медицинские статистические данные и геопространственную информацию, что позволяет визуализировать пространственное распределение уровней риска заболеваемости мочекаменной болезнью на территории Воронежской области.

В ходе выполнения работы были решены следующие задачи:

- 1) проведён анализ существующих инструментов и библиотек для создания интерактивных веб-карт, обоснован выбор Leaflet.js в качестве основного средства визуализации;
- 2) разработана процедура сбора и предобработки исходных атрибутивных данных (статистические показатели заболеваемости) с использованием языка Python и библиотеки pandas;
- 3) выполнен парсинг и верификация геопространственных границ районов Воронежской области, получен корректный GeoJSON-файл;
- 4) реализован программный модуль на Python, выполняющий обработку данных, расчёт прогнозных значений и генерацию HTML-страницы со встроенным кодом и данными;
- 5) разработана клиентская часть на JavaScript с использованием библиотеки Leaflet.js, обеспечивающая отображение карты, цветовую кодировку районов, всплывающие подсказки, информационные окна и фильтрацию данных через боковую панель;
- 6) проведено тестирование разработанного модуля на персональном компьютере, подтвердившее его работоспособность и корректность всех интерактивных элементов;
- 7) оценена масштабируемость модуля и показана возможность его адаптации для других регионов без изменения программного кода.

Разработанный инструмент может быть использован региональным министерством здравоохранения для кластеризации зон риска, планирования

выделения врачей-урологов, закупки оборудования и организации профилактических мероприятий. Архитектура модуля является универсальной и может быть адаптирована для визуализации медико-статистических данных любых регионов Российской Федерации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Калинин, П. В. Пространственный анализ в геоинформатике [Текст] / П. В. Калинин // Цифровая обработка сигналов. — 2013. — № 1. — С. 28-33.
2. Борисов, В. В. Медицинская география и урологическая заболеваемость в Центральном Черноземье [Текст] / В. В. Борисов, В. А. Ковалев. — Воронеж : Научная книга, 2021. — 210 с.
3. Leaflet.js – документация [Электронный ресурс]. — URL: <https://leafletjs.com/reference.html> (дата обращения: 01.06.2024). — Текст : электронный.
4. ГОСТ Р 7.0.100-2018. Библиографическая запись. Библиографическое описание. Общие требования и правила составления : национальный стандарт Российской Федерации : дата введения 2019-07-01 / Федеральное агентство по техническому регулированию. — Изд. официальное. — Москва : Стандартинформ, 2018. — 124 с.
5. Дорохова, С. А. Разработка интерактивной карты прогноза заболеваемости мочекаменной болезнью в Воронежской области [Электронный ресурс] / С. А. Дорохова, А. И. Борисов, Е. А. Сирота // Вестник студенческой науки факультета компьютерных наук ВГУ. — 2026. — № 6. — С. 665-670. — URL: <https://www.cs.vsu.ru/stwork/conf/2026/vol-pro-6-2026.pdf> (дата обращения: 11.06.2026). — Текст : электронный.

ПРИЛОЖЕНИЕ А

Листинг А.1 – Пример Feature-объекта GeoJSON

```
{
  "type": "Feature",
  "geometry": {
    "type": "Polygon",
    "coordinates": [...]
  },
  "properties": {
    "name": "Богучарский район",
    "total_morbidity": 2801.8,
    "level": "Высокий"
  }
}
```

Листинг А.2 – Подготовка исходных данных и классификация уровней

```
import json
from pathlib import Path
import webbrowser

# ===== ПРОГНОЗНЫЕ ДАННЫЕ ИЗ ДИССЕРТАЦИИ =====
mkb_forecast_2026 = {
    'Аннинский': 1234.9, 'Бобровский': 1102.2, 'Богучарский': 2801.8,
    'Борисоглебский': 963.2, 'Бутурлиновский': 159.0, 'Верхнемамонский':
788.9,
    'Верхнехавский': 445.8, 'Воробьевский': 165.3, 'Грибановский': 1131.6,
    'Калачеевский': 301.8, 'Каменский': 2323.1, 'Кантемировский': 1219.0,
    'Каширский': 1695.5, 'Лискинский': 1072.6, 'Нижедевицкий': 2272.4,
    'Новоусманский': 862.4, 'Новохоперский': 470.8, 'Ольховатский': 814.0,
    'Острогожский': 334.2, 'Павловский': 2795.9, 'Панинский': 1201.7,
    'Петропавловский': 1024.5, 'Поворинский': 299.8, 'Подгоренский': 609.4,
    'Рамонский': 992.1, 'Репьевский': 2746.8, 'Россошанский': 747.9,
    'Семилукский': 713.6, 'Таловский': 524.5, 'Терновский': 942.6,
    'Хохольский': 976.9, 'Эртильский': 802.3,
    # ДОБАВЛЯЕМ ГОРОДА
    'Воронеж': 950.0,
    'Нововоронеж': 850.0
}

primary_forecast_2026 = {
    'Аннинский': 85.9, 'Бобровский': 176.4, 'Богучарский': 1901.7,
    'Борисоглебский': 349.4, 'Бутурлиновский': 34.2, 'Верхнемамонский':
123.8,
    'Верхнехавский': 100.1, 'Воробьевский': -10.5, 'Грибановский': 62.2,
    'Калачеевский': 116.9, 'Каменский': 12.5, 'Кантемировский': 67.6,
    'Каширский': 227.0, 'Лискинский': 81.8, 'Нижедевицкий': 234.6,
    'Новоусманский': 3.0, 'Новохоперский': 39.8, 'Ольховатский': 121.9,
    'Острогожский': 13.8, 'Павловский': 829.1, 'Панинский': 110.3,
    'Петропавловский': 209.1, 'Поворинский': 335.0, 'Подгоренский': 393.1,
    'Рамонский': 132.8, 'Репьевский': 190.4, 'Россошанский': 85.5,
    'Семилукский': 277.5, 'Таловский': -31.4, 'Терновский': 114.0,
    'Хохольский': -13.2, 'Эртильский': 53.8,
    'Воронеж': 380.0,
    'Нововоронеж': 420.0
}

dispensary_forecast_2026 = {
```



```

'Аннинский': 1130.1, 'Бобровский': 765.1, 'Богучарский': 1775.3,
'Борисоглебский': 710.0, 'Бутурлиновский': 94.5, 'Верхнемамонский':
734.7,
'Верхнехавский': 365.1, 'Воробьевский': 165.5, 'Грибановский': 1053.6,
'Калачеевский': 316.5, 'Каменский': 2230.0, 'Кантемировский': 1055.3,
'Каширский': 1458.3, 'Лискинский': 989.7, 'Нижедевицкий': 2243.4,
'Новоусманский': 837.3, 'Новохоперский': 457.4, 'Ольховатский': 460.7,
'Острогожский': 249.4, 'Павловский': 2068.3, 'Панинский': 1066.7,
'Петропавловский': 837.1, 'Поворинский': 147.6, 'Подгоренский': 44.4,
'Рамонский': 1009.6, 'Репьевский': 2293.0, 'Россошанский': 84.6,
'Семилукский': 377.2, 'Таловский': 445.6, 'Терновский': 942.6,
'Хохольский': 1079.8, 'Эртильский': 703.3,
'Воронеж': 890.0,
'Нововоронеж': 780.0
}

LEVEL_LOW_MAX = 438.3
LEVEL_MEDIUM_MIN = 438.3
LEVEL_MEDIUM_MAX = 1075.1
LEVEL_HIGH_MIN = 1075.1

# Причины заболеваемости для каждого района
reasons = {
    'Высокий': {
        'Каменский': 'Высокая минерализация воды, отсутствие врачей-урологов,
удаленность от областного центра',
        'Павловский': 'Жесткая вода, недостаточная диспансеризация, высокая
доля пожилого населения',
        'Репьевский': 'Критическая ситуация: экстремально жесткая вода,
отсутствие медицинской инфраструктуры',
        'Богучарский': 'Геологические особенности почвы, высокая
минерализация подземных вод',
        'Нижедевицкий': 'Отдаленность от урологических центров, низкая
доступность специализированной помощи',
        'Каширский': 'Промышленные загрязнения воды, недостаток врачей-
урологов',
        'Аннинский': 'Высокое содержание солей в воде, недостаточная
профилактическая работа',
        'Кантемировский': 'Природно-климатические факторы, особенности
питания населения',
        'Панинский': 'Неблагоприятная экологическая обстановка, недостаток
чистой питьевой воды',
        'Грибановский': 'Высокий уровень урбанизации, стрессовые факторы',
        'Бобровский': 'Промышленные выбросы, качество питьевой воды'
    },
    'Средний': {
        'Лискинский': 'Средний уровень минерализации, доступность медицинской
помощи',
        'Петропавловский': 'Умеренное содержание солей, удовлетворительная
диспансеризация',
        'Рамонский': 'Близость к областному центру, доступность урологической
помощи',
        'Хохольский': 'Сбалансированный состав воды, наличие врачей',
        'Борисоглебский': 'Городская среда, доступность диагностики',
        'Терновский': 'Умеренные показатели, требуется усиление
профилактики',
        'Новоусманский': 'Близость к Воронежу, хорошая медицинская
доступность',
        'Эртильский': 'Средние показатели, требуется мониторинг',
        'Верхнемамонский': 'Удаленность, но удовлетворительная ситуация',
        'Ольховатский': 'Средний уровень, требуется профилактика',
        'Россошанский': 'Доступность медицинской помощи, средние показатели',
        'Семилукский': 'Близость к городу, удовлетворительная ситуация',
    }
}

```

```

        'Подгоренский': 'Требуется улучшение диспансеризации',
        'Таловский': 'Средние показатели, стабильная ситуация',
        'Воронеж': 'Крупный город, хорошая медицинская инфраструктура,
доступность урологической помощи',
        'Нововоронеж': 'Город с развитой медициной, наличие специалистов'
    },
    'Низкий': {
        'Новохоперский': 'Хорошее качество воды, развитая профилактика',
        'Верхнехавский': 'Низкая минерализация, хорошая медицинская
доступность',
        'Острогожский': 'Благоприятная экология, качественная питьевая вода',
        'Калачеевский': 'Низкое содержание солей, хорошая профилактика',
        'Поворинский': 'Экологически чистый район, мягкая вода',
        'Воробьевский': 'Низкий уровень заболеваемости, благоприятные
факторы',
        'Бутурлиновский': 'Хорошие природные условия, доступность медицины'
    }
}

```

Листинг А.3 – Функции обработки и загрузки геоданных

```

def get_color(morbidity):
    if morbidity >= LEVEL_HIGH_MIN:
        return '#8B0000'
    elif morbidity >= LEVEL_MEDIUM_MIN:
        return '#FFA500'
    else:
        return '#90EE90'

def get_level(morbidity):
    if morbidity >= LEVEL_HIGH_MIN:
        return 'Высокий'
    elif morbidity >= LEVEL_MEDIUM_MIN:
        return 'Средний'
    else:
        return 'Низкий'

def load_geojson():
    """Загружает GeoJSON данные из файла"""
    # Пробуем разные варианты имени файла
    possible_names = [
        'Воронежская область_Voronezh.txt',
        'Voronezh_obl.geojson.txt',
        'voronezh_districts.geojson'
    ]

    for geojson_file in possible_names:
        file_path = Path(geojson_file)
        if file_path.exists():
            with open(file_path, 'r', encoding='utf-8') as f:
                print(f"✅ Найден файл: {geojson_file}")
                return json.load(f)

    return None

# Создаем данные для каждого района
districts_info = {}
for district, morbidity in mkb_forecast_2026.items():
    level = get_level(morbidity)
    reason_dict = reasons.get(level, {})
    districts_info[district] = {
        'name': district,
        'total_morbidity': morbidity,

```

```

        'primary_morbidity': max(0, primary_forecast_2026.get(district, 0)),
        'dispensary': dispensary_forecast_2026.get(district, 0),
        'level': level,
        'level_full': f'{level} ({morbidity:.1f})',
        'color': get_color(morbidity),
        'reason': reason_dict.get(district, 'Требуется дополнительное
исследование факторов риска')
    }

values = list(mkb_forecast_2026.values())
high_count = sum(1 for v in values if v >= LEVEL_HIGH_MIN)
medium_count = sum(1 for v in values if LEVEL_MEDIUM_MIN <= v <
LEVEL_MEDIUM_MAX)
low_count = sum(1 for v in values if v < LEVEL_LOW_MAX)

# Загружаем GeoJSON
geojson_data = load_geojson()

print("\n" + "=" * 60)
print("📊 КЛАССИФИКАЦИЯ РАЙОНОВ ПО ПРОГНОЗУ НА 2026 ГОД")
print("=" * 60)
print(f"🔴 Высокий уровень (>1075.1): {high_count} районов")
print(f"🟡 Средний уровень (438.3-1075.1): {medium_count} районов")
print(f"🟢 Низкий уровень (<438.3): {low_count} районов")
print(f"📍 Воронеж: {get_level(950.0)} уровень (950.0)")
print(f"📍 Нововоронеж: {get_level(850.0)} уровень (850.0)")
print("=" * 60)

```

Листинг А.4 – Клиентская часть на JavaScript (Leaflet.js)

```

const districtsData = {json.dumps(districts_info, ensure_ascii=False)};
let geojsonLayer = null;
let currentHighlightedLayer = null;

function switchTab(tabName) {{
    document.querySelectorAll('.tab-content').forEach(tab =>
tab.classList.remove('active'));
    document.querySelectorAll('.tab-btn').forEach(btn =>
btn.classList.remove('active'));
    document.getElementById(tabName + '-tab').classList.add('active');
    event.target.classList.add('active');
}}

function showDistrictInfo(districtName) {{
    const data = districtsData[districtName];
    if (!data) return;

    let levelClass = '';
    if (data.level === 'Высокий') levelClass = 'critical';
    else if (data.level === 'Средний') levelClass = 'medium';
    else levelClass = 'low';

    let displayName = districtName;
    if (!districtName.includes('Воронеж')) {{
        displayName = districtName + ' район';
    }}

    document.getElementById('district-detail').innerHTML = `
        <h4>📍 ${{displayName}}</h4>

```

```

        <div class="info-row"><span class="info-label">Уровень:</span><span
class="info-value ${levelClass}">${data.level}}
(${data.total_morbidity.toFixed(1)})</span></div>
        <div class="info-row"><span class="info-label">Прогноз общей
заболеваемости (2026):</span><span class="info-
value">${data.total_morbidity.toFixed(1)} на 100 тыс.</span></div>
        <div class="info-row"><span class="info-label">Прогноз первичной
заболеваемости:</span><span class="info-
value">${data.primary_morbidity.toFixed(1)} на 100 тыс.</span></div>
        <div class="info-row"><span class="info-label">Прогноз диспансерного
учета:</span><span class="info-value">${data.dispensary.toFixed(1)} на 100
тыс.</span></div>
        <div class="reason-text">
            <strong>🔍 Вероятные причины:</strong><br>
            ${data.reason}}
        </div>
    `;

    // Переключаемся на вкладку информации при клике
    document.querySelectorAll('.tab-content').forEach(tab =>
tab.classList.remove('active'));
    document.querySelectorAll('.tab-btn').forEach(btn =>
btn.classList.remove('active'));
    document.getElementById('info-tab').classList.add('active');
    document.querySelector('.tab-btn').classList.add('active');
}

function matchDistrictName(geoName) {{
    let cleanName = geoName.replace(/ район$/i, '').replace(/ёвский/i,
'евский');
    if (districtsData[cleanName]) return cleanName;
    for (const key of Object.keys(districtsData)) {{
        if (cleanName.includes(key) || key.includes(cleanName)) return key;
    }}
    return null;
}}

function styleFeature(feature) {{
    const geoName = feature.properties?.district || feature.properties?.name
|| '';
    const matchedKey = matchDistrictName(geoName);
    const fillColor = matchedKey ? districtsData[matchedKey].color :
'#CCCCCC';
    return {{
        fillColor: fillColor,
        color: '#ffffff',
        weight: 1.5,
        opacity: 0.8,
        fillOpacity: 0.65
    }};
}}

function highlightFeature(e) {{
    const layer = e.target;
    if (currentHighlightedLayer && currentHighlightedLayer !== layer) {{
        geojsonLayer.resetStyle(currentHighlightedLayer);
    }}
    layer.setStyle({{
        weight: 3,
        color: '#ffff00',
        fillOpacity: 0.9
    }});
    currentHighlightedLayer = layer;

```

```

        if (!L.Browser.ie && !L.Browser.opera) {{
            layer.bringToFront();
        }}
    }}

function resetHighlight(e) {{
    if (geojsonLayer) {{
        geojsonLayer.resetStyle(e.target);
    }}
    currentHighlightedLayer = null;
}}

function onFeatureClick(e) {{
    const feature = e.target.feature;
    const geoName = feature.properties?.district || feature.properties?.name
    || '';
    const matchedKey = matchDistrictName(geoName);
    if (matchedKey) {{
        showDistrictInfo(matchedKey);
    }}
}}

function onEachFeature(feature, layer) {{
    const geoName = feature.properties?.district || feature.properties?.name
    || '';
    const matchedKey = matchDistrictName(geoName);
    if (matchedKey && districtsData[matchedKey]) {{
        const data = districtsData[matchedKey];
        let displayName = matchedKey.includes('Воронеж') ? matchedKey :
matchedKey + ' район';
        layer.bindTooltip(`
            <b>${{displayName}}</b><br>
            Прогноз: ${{data.total_morbidity.toFixed(1)}}<br>
            Уровень: ${{data.level}}
        `, {{ sticky: true, direction: 'top' }});
    }}
    layer.on({{
        mouseover: highlightFeature,
        mouseout: resetHighlight,
        click: onFeatureClick
    }});
}}

const map = L.map('map').setView([51.2, 40.2], 8);
L.tileLayer('https://{{s}}.basemaps.cartocdn.com/light_all/{{z}}/{{x}}/{{y}}{{
{r}}}.png', {{
    attribution: '',
    subdomains: 'abcd',
    maxZoom: 19
}}).addTo(map);

const geojsonData = {json.dumps(geojson_data) if geojson_data else 'null'};

if (geojsonData) {{
    geojsonLayer = L.geoJson(geojsonData, {{
        style: styleFeature,
        onEachFeature: onEachFeature
    }}).addTo(map);
    map.fitBounds(geojsonLayer.getBounds());
}} else {{
    document.getElementById('district-detail').innerHTML = `
        <h4>⚠ Ошибка</h4>
        <p style="color:#999;">Файл с границами районов не найден.<br>

```

```

        Убедитесь, что файл <strong>Воронежская область_Voronezh.txt</strong>
        находится в той же папке.</p>
        `;
    }}

function buildDistrictList() {{
    const districts = Object.entries(districtsData);
    const highDistricts = districts.filter(d => d[1].level ===
'Высокий').sort((a,b) => b[1].total_morbidity - a[1].total_morbidity);
    const mediumDistricts = districts.filter(d => d[1].level ===
'Средний').sort((a,b) => b[1].total_morbidity - a[1].total_morbidity);
    const lowDistricts = districts.filter(d => d[1].level ===
'Низкий').sort((a,b) => b[1].total_morbidity - a[1].total_morbidity);

    let html = '';
    if (highDistricts.length > 0) {{
        html += '<h4 style="color:#8B0000; margin:10px 0 5px 0;">● Высокий
уровень</h4>';
        highDistricts.forEach(([name, data]) => {{
            let displayName = name.includes('Воронеж') ? name : name + '
район';
            html += `<div class="district-item district-high"
onclick="showDistrictInfo('${{name}}')">
                <strong>${{displayName}}</strong> -
                ${{data.total_morbidity.toFixed(1)}} на 100 тыс.
                </div>`;
            }});
    }}
    if (mediumDistricts.length > 0) {{
        html += '<h4 style="color:#FFA500; margin:15px 0 5px 0;">□ Средний
уровень</h4>';
        mediumDistricts.forEach(([name, data]) => {{
            let displayName = name.includes('Воронеж') ? name : name + '
район';
            html += `<div class="district-item district-medium"
onclick="showDistrictInfo('${{name}}')">
                <strong>${{displayName}}</strong> -
                ${{data.total_morbidity.toFixed(1)}} на 100 тыс.
                </div>`;
            }});
    }}
    if (lowDistricts.length > 0) {{
        html += '<h4 style="color:#228B22; margin:15px 0 5px 0;">□ Низкий
уровень</h4>';
        lowDistricts.forEach(([name, data]) => {{
            let displayName = name.includes('Воронеж') ? name : name + '
район';
            html += `<div class="district-item district-low"
onclick="showDistrictInfo('${{name}}')">
                <strong>${{displayName}}</strong> -
                ${{data.total_morbidity.toFixed(1)}} на 100 тыс.
                </div>`;
            }});
    }}
    document.getElementById('district-list').innerHTML = html;
}}

buildDistrictList();

// Показываем первый район
const firstDistrict = Object.keys(districtsData)[0];
if (firstDistrict) setTimeout(() => showDistrictInfo(firstDistrict), 500);

```

Листинг А.5 – Генерация динамической HTML-страницы на Python

```
# ===== СОЗДАНИЕ HTML КАРТЫ =====
html_content = f'''<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Прогноз заболеваемости МКБ в Воронежской области на 2026
род</title>
    <link rel="stylesheet"
href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" />
    <script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
    <style>
        * {{ margin: 0; padding: 0; box-sizing: border-box; }}
        body {{
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            min-height: 100vh;
        }}
        .header {{
            background: rgba(0,0,0,0.8);
            color: white;
            padding: 20px;
            text-align: center;
        }}
        .header h1 {{ font-size: 28px; margin-bottom: 5px; }}
        .header p {{ font-size: 14px; opacity: 0.8; }}
        .container {{ display: flex; padding: 20px; gap: 20px; max-width:
1600px; margin: 0 auto; }}
        .map-container {{ flex: 2; background: white; border-radius: 15px;
overflow: hidden; box-shadow: 0 5px 20px rgba(0,0,0,0.3); }}
        #map {{ height: 700px; width: 100%; }}
        .info-panel {{
            flex: 1; background: white; border-radius: 15px; padding: 20px;
            box-shadow: 0 5px 20px rgba(0,0,0,0.3); max-height: 700px;
overflow-y: auto;
            display: flex;
            flex-direction: column;
            gap: 20px;
        }}
        .legend {{ background: #f8f9fa; padding: 15px; border-radius: 10px;
}}
        .legend h3 {{ margin-bottom: 10px; color: #333; font-size: 16px; }}
        .legend-item {{ display: flex; align-items: center; margin-bottom:
8px; }}
        .legend-color {{ width: 30px; height: 20px; margin-right: 10px;
border-radius: 3px; }}

        .tabs {{
            display: flex;
            gap: 5px;
            margin-bottom: 15px;
        }}
        .tab-btn {{
            flex: 1;
            padding: 10px;
            border: none;
            background: #f0f0f0;
            cursor: pointer;
            border-radius: 8px;

```

```

        font-weight: bold;
        transition: all 0.3s;
    }}
    .tab-btn.active {{
        background: #667eea;
        color: white;
    }}
    .tab-content {{
        display: none;
    }}
    .tab-content.active {{
        display: block;
    }}

    .district-list {{
        max-height: 300px;
        overflow-y: auto;
    }}
    .district-item {{
        padding: 10px;
        margin: 5px 0;
        border-radius: 8px;
        cursor: pointer;
        transition: all 0.2s;
        border-left: 4px solid;
    }}
    .district-item:hover {{
        transform: translateX(5px);
        background: #f0f0f0;
    }}
    .district-high {{ border-left-color: #8B0000; background: #FFF5F5; }}
    .district-medium {{ border-left-color: #FFA500; background: #FFF8F0; }}
}}

    .district-low {{ border-left-color: #90EE90; background: #F0FFF0; }}

    .district-detail {{
        background: #f8f9fa;
        padding: 15px;
        border-radius: 10px;
        margin-top: 10px;
    }}
    .district-detail h4 {{ margin-bottom: 10px; color: #333; }}
    .reason-text {{
        background: #fff;
        padding: 12px;
        border-radius: 8px;
        margin-top: 10px;
        font-size: 14px;
        line-height: 1.5;
        color: #555;
        border-left: 4px solid #667eea;
    }}
    .statistics {{ background: #f8f9fa; padding: 15px; border-radius:
10px; }}
    .info-row {{ display: flex; justify-content: space-between; padding:
8px 0; border-bottom: 1px solid #e0e0e0; }}
    .info-label {{ font-weight: bold; color: #555; }}
    .critical {{ color: #8B0000; font-weight: bold; }}
    .medium {{ color: #FFA500; font-weight: bold; }}
    .low {{ color: #228B22; font-weight: bold; }}

    footer {{ text-align: center; padding: 15px; background:
rgba(0,0,0,0.8); color: white; font-size: 12px; }}

```



```

        @media (max-width: 900px) {{ .container {{ flex-direction: column; }}
.info-panel {{ max-height: 400px; }} }}
        .leaflet-control-attribution {{ display: none !important; }}
    </style>
</head>
<body>
    <div class="header">
        <h1><img alt="map icon" data-bbox="265 165 285 180"/> Прогноз заболеваемости мочекаменной болезнью (МКБ) в
Воронежской области</h1>
        <p>Прогнозные данные на 2026 год | Показатели на 100 000 населения |
Интерактивная карта с подсветкой районов</p>
    </div>
    <div class="container">
        <div class="map-container"><div id="map"></div></div>
        <div class="info-panel">
            <div class="legend">
                <h3><img alt="legend icon" data-bbox="345 290 365 305"/> Классификация районов</h3>
                <div class="legend-item"><div class="legend-color"
style="background:#8B0000"></div><span>Высокий уровень (>1075,1)</span></div>
                <div class="legend-item"><div class="legend-color"
style="background:#FFA500"></div><span>Средний уровень (438,3-
1075,1)</span></div>
                <div class="legend-item"><div class="legend-color"
style="background:#90EE90"></div><span>Низкий уровень (<438,3)</span></div>
            </div>

            <div class="tabs">
                <button class="tab-btn active" onclick="switchTab('info')"><img alt="info icon" data-bbox="900 435 920 450"/>
Информация</button>
                <button class="tab-btn" onclick="switchTab('list')"><img alt="list icon" data-bbox="825 465 845 480"/> Список
районов</button>
            </div>

            <div id="info-tab" class="tab-content active">
                <div id="district-detail" class="district-detail">
                    <h4><img alt="district icon" data-bbox="385 550 405 565"/> Выберите район</h4>
                    <p style="color:#999;">Нажмите на район на карте или
выберите из списка</p>
                    <p style="color:#999; margin-top:10px;"><img alt="lightbulb icon" data-bbox="750 595 765 610"/> При наведении
на район он подсвечивается</p>
                </div>
            </div>

            <div id="list-tab" class="tab-content">
                <div class="district-list" id="district-list">
                    <!-- Список районов будет заполнен через JS -->
                </div>
            </div>

            <div class="statistics">
                <h3><img alt="checkbox icon" data-bbox="345 755 365 770"/> Прогнозная статистика на 2026 год</h3>
                <div class="info-row"><span class="info-label">Районов с
высоким уровнем:</span><span class="info-value
critical">{high_count}</span></div>
                <div class="info-row"><span class="info-label">Районов со
средним уровнем:</span><span class="info-value
medium">{medium_count}</span></div>
                <div class="info-row"><span class="info-label">Районов с
низким уровнем:</span><span class="info-value low">{low_count}</span></div>
                <div class="info-row"><span class="info-
label">Воронеж:</span><span class="info-value medium">Средний
(950.0)</span></div>

```

```

        <div class="info-row"><span class="info-label">Нововоронеж:</span><span class="info-value medium">Средний
(850.0)</span></div>
    </div>
</div>
<footer>Источник: диссертация, Глава 3, Таблицы 3.4, 3.5, 3.6, 3.7 |
Границы районов: OpenStreetMap</footer>

<script>
    const districtsData = {json.dumps(districts_info,
ensure_ascii=False)};
    let geojsonLayer = null;
    let currentHighlightedLayer = null;

    function switchTab(tabName) {{
        document.querySelectorAll('.tab-content').forEach(tab =>
tab.classList.remove('active'));
        document.querySelectorAll('.tab-btn').forEach(btn =>
btn.classList.remove('active'));
        document.getElementById(tabName + '-
tab').classList.add('active');
        event.target.classList.add('active');
    }}

    function showDistrictInfo(districtName) {{
        const data = districtsData[districtName];
        if (!data) return;

        let levelClass = '';
        if (data.level === 'Высокий') levelClass = 'critical';
        else if (data.level === 'Средний') levelClass = 'medium';
        else levelClass = 'low';

        let displayName = districtName;
        if (!districtName.includes('Воронеж')) {{
            displayName = districtName + ' район';
        }}

        document.getElementById('district-detail').innerHTML = `
            <h4>🏠 ${{displayName}}</h4>
            <div class="info-row"><span class="info-label">Уровень:</span><span class="info-value
${{levelClass}}">${{data.level}}
(${{data.total_morbidity.toFixed(1)}})</span></div>
            <div class="info-row"><span class="info-label">Прогноз общей
заболеваемости (2026):</span><span class="info-value">${{data.total_morbidity.toFixed(1)}} на 100 тыс.</span></div>
            <div class="info-row"><span class="info-label">Прогноз
первичной заболеваемости:</span><span class="info-value">${{data.primary_morbidity.toFixed(1)}} на 100 тыс.</span></div>
            <div class="info-row"><span class="info-label">Прогноз
диспансерного учета:</span><span class="info-value">${{data.dispensary.toFixed(1)}} на 100 тыс.</span></div>
            <div class="reason-text">
                <strong>🔍 Вероятные причины:</strong><br>
                ${{data.reason}}
            </div>
        `;

        // Переключаемся на вкладку информации при клике
        document.querySelectorAll('.tab-content').forEach(tab =>
tab.classList.remove('active'));

```

```

        document.querySelectorAll('.tab-btn').forEach(btn =>
        btn.classList.remove('active'));
        document.getElementById('info-tab').classList.add('active');
        document.querySelector('.tab-btn').classList.add('active');
    })

    function matchDistrictName(geoName) {{
        let cleanName = geoName.replace(/ район$/i,
        '').replace(/ёвский/i, 'евский');
        if (districtsData[cleanName]) return cleanName;
        for (const key of Object.keys(districtsData)) {{
            if (cleanName.includes(key) || key.includes(cleanName))
return key;
        }}
        return null;
    }}

    function styleFeature(feature) {{
        const geoName = feature.properties?.district ||
feature.properties?.name || '';
        const matchedKey = matchDistrictName(geoName);
        const fillColor = matchedKey ? districtsData[matchedKey].color :
'#CCCCCC';
        return {{
            fillColor: fillColor,
            color: '#ffffff',
            weight: 1.5,
            opacity: 0.8,
            fillOpacity: 0.65
        }};
    }}

    function highlightFeature(e) {{
        const layer = e.target;
        if (currentHighlightedLayer && currentHighlightedLayer !== layer)
{{
            geojsonLayer.resetStyle(currentHighlightedLayer);
        }}
        layer.setStyle({{
            weight: 3,
            color: '#ffff00',
            fillOpacity: 0.9
        }});
        currentHighlightedLayer = layer;
        if (!L.Browser.ie && !L.Browser.opera) {{
            layer.bringToFront();
        }}
    }}

    function resetHighlight(e) {{
        if (geojsonLayer) {{
            geojsonLayer.resetStyle(e.target);
        }}
        currentHighlightedLayer = null;
    }}

    function onFeatureClick(e) {{
        const feature = e.target.feature;
        const geoName = feature.properties?.district ||
feature.properties?.name || '';
        const matchedKey = matchDistrictName(geoName);
        if (matchedKey) {{
            showDistrictInfo(matchedKey);
        }}
    }}

```

```

    }}
  }}

  function onEachFeature(feature, layer) {{
    const geoName = feature.properties?.district ||
feature.properties?.name || '';
    const matchedKey = matchDistrictName(geoName);
    if (matchedKey && districtsData[matchedKey]) {{
      const data = districtsData[matchedKey];
      let displayName = matchedKey.includes('Воронеж') ? matchedKey
: matchedKey + ' район';
      layer.bindTooltip(`
        <b>${{displayName}}</b><br>
        Прогноз: ${{data.total_morbidity.toFixed(1)}}<br>
        Уровень: ${{data.level}}
      `, {{ sticky: true, direction: 'top' }});
    }}
    layer.on({{
      mouseover: highlightFeature,
      mouseout: resetHighlight,
      click: onFeatureClick
    }});
  }}

  const map = L.map('map').setView([51.2, 40.2], 8);

  L.tileLayer('https://{{s}}.basemaps.cartocdn.com/light_all/{{z}}/{{x}}/{{y}}{
{r}}.png', {{
    attribution: '',
    subdomains: 'abcd',
    maxZoom: 19
  }}).addTo(map);

  const geojsonData = {json.dumps(geojson_data) if geojson_data else
'null'};

  if (geojsonData) {{
    geojsonLayer = L.geoJson(geojsonData, {{
      style: styleFeature,
      onEachFeature: onEachFeature
    }}).addTo(map);
    map.fitBounds(geojsonLayer.getBounds());
  }} else {{
    document.getElementById('district-detail').innerHTML = `
      <h4>⚠ Ошибка</h4>
      <p style="color:#999;">Файл с границами районов не
найден.<br>
      Убедитесь, что файл <strong>Воронежская
область_Voronezh.txt</strong> находится в той же папке.</p>
    `;
  }}

  function buildDistrictList() {{
    const districts = Object.entries(districtsData);
    const highDistricts = districts.filter(d => d[1].level ===
'Высокий').sort((a,b) => b[1].total_morbidity - a[1].total_morbidity);
    const mediumDistricts = districts.filter(d => d[1].level ===
'Средний').sort((a,b) => b[1].total_morbidity - a[1].total_morbidity);
    const lowDistricts = districts.filter(d => d[1].level ===
'Низкий').sort((a,b) => b[1].total_morbidity - a[1].total_morbidity);

    let html = '';
    if (highDistricts.length > 0) {{

```

```

        html += '<h4 style="color:#8B0000; margin:10px 0 5px 0;">●
Высокий уровень</h4>';
        highDistricts.forEach(([name, data]) => {{
            let displayName = name.includes('Воронеж') ? name : name
+ ' район';
            html += `<div class="district-item district-high"
onclick="showDistrictInfo('${{name}}')">
                <strong>${{displayName}}</strong> –
${{data.total_morbidity.toFixed(1)}} на 100 тыс.
                </div>`;
            }});
    }}
    if (mediumDistricts.length > 0) {{
        html += '<h4 style="color:#FFA500; margin:15px 0 5px 0;">□
Средний уровень</h4>';
        mediumDistricts.forEach(([name, data]) => {{
            let displayName = name.includes('Воронеж') ? name : name
+ ' район';
            html += `<div class="district-item district-medium"
onclick="showDistrictInfo('${{name}}')">
                <strong>${{displayName}}</strong> –
${{data.total_morbidity.toFixed(1)}} на 100 тыс.
                </div>`;
            }});
    }}
    if (lowDistricts.length > 0) {{
        html += '<h4 style="color:#228B22; margin:15px 0 5px 0;">□
Низкий уровень</h4>';
        lowDistricts.forEach(([name, data]) => {{
            let displayName = name.includes('Воронеж') ? name : name
+ ' район';
            html += `<div class="district-item district-low"
onclick="showDistrictInfo('${{name}}')">
                <strong>${{displayName}}</strong> –
${{data.total_morbidity.toFixed(1)}} на 100 тыс.
                </div>`;
            }});
    }}
    document.getElementById('district-list').innerHTML = html;
}}

buildDistrictList();

// Показываем первый район
const firstDistrict = Object.keys(districtsData)[0];
if (firstDistrict) setTimeout(() => showDistrictInfo(firstDistrict),
500);
</script>
</body>
</html>
'''

output_file = Path('voronezh_mkb_forecast_2026.html')
output_file.write_text(html_content, encoding='utf-8')

print(f"\n☑ Карта сохранена: {output_file.absolute()}")
print("\n🔗 Добавленные функции:")
print("    • 🌟 Подсветка районов при наведении (жёлтая обводка)")
print("    • 🗺 Отображение границ районов с цветовой заливкой")
print("    • 💬 Всплывающие подсказки при наведении")
print("    • 📍 Клик по району показывает подробную информацию")

```

```

print("    • 🗺️ Добавлены города Воронеж и Нововоронеж (средний уровень)")

# Проверка наличия GeoJSON файла
geojson_file = Path('Воронежская область_Voronezh.txt')
if not geojson_file.exists():
    print(f"\n⚠️ ВНИМАНИЕ: Файл {geojson_file} не найден!")
    print("    Карта не сможет отобразить границы районов.")
    print("    Убедитесь, что файл находится в той же папке.")
else:
    print(f"\n✅ Файл с границами районов найден: {geojson_file}")

try:
    webbrowser.open(str(output_file.absolute()))
    print(f"\n🌐 Файл автоматически открыт в браузере!")
except:
    pass

```